

Vision Task Report

A Performance Analysis of Deep Learning Architectures for the classification of 15 Selected Fruit Varieties using CNNs and Transfer Learning

6CS012

Artificial Intelligence and Machine Learning

Herald College Kathmandu

Student Information:

Name	Student Number	Group	Date
Pratik Karki	2408034	L6CG15	May 10, 2026

Abstract

This report helps to evaluate how effective Convolutional Neural Networks (CNNs) are by classifying 15 different fruits from a subset of fruits-360 dataset. We worked with Baseline CNN, Deeper CNN with regularization and explored transfer learning. Using augmentation, each training dataset was capped to 1000 images per class which makes 15000 images in total. Results show that baseline model. The study provides an analysis between SGD and Adam with computational efficiency metrics. The key finding of this analysis is that despite deeper models and transfer learning having better performance, they have high computational costs. This research highlights practical trade-offs between model's complexity, accuracy and training efficiency.

Table Of Content

1. Introduction.....	4
Dataset.....	4
Methodology.....	5
Baseline CNN architecture.....	5
Deeper CNN Architecture.....	5
Transfer Learning Model.....	6
Experiments and Results.....	6
Baseline vs Deeper Architecture.....	6
Comparison Between Baseline and Deeper Architecture.....	6
Computational Efficiency.....	7
Optimizer Analysis: SGD and Adam.....	7
Ablation Study.....	7
Challenges in Training.....	8
Hyperparameter tuning.....	8
Fine-Tuning and Transfer learning.....	8
Conclusion.....	9
Key Finding.....	9
Limitations.....	9
Future Improvements.....	9

1. Introduction

In deep learning, classifying images is one of important applications. Automatically categorizing images into predefined classes using machine learning algorithms. Convolutional Neural Networks (CNNs) are a popular method to classify images as they can automatically learn hierarchical visual features from image data.

This project analyzes the performance of CNN models for fruit image classification. First, building a baseline CNN architecture from scratch then extending the model into deeper architecture and regularization techniques. Then at last, using transfer learning methods to use pre-trained models like MobileNet to compare its performance with CNN models.

The main objective of this project is to analyze and preprocess fruit image dataset, build and train baseline CNN model, implement deeper CNN models, compare Adam and SGD optimizers, apply transfer learning and compare the performance between CNNs and transfer learning.

This project also highlights the advantages of transfer learning in real-world applications with the drawbacks like model complexity, training time and classification performance.

Dataset

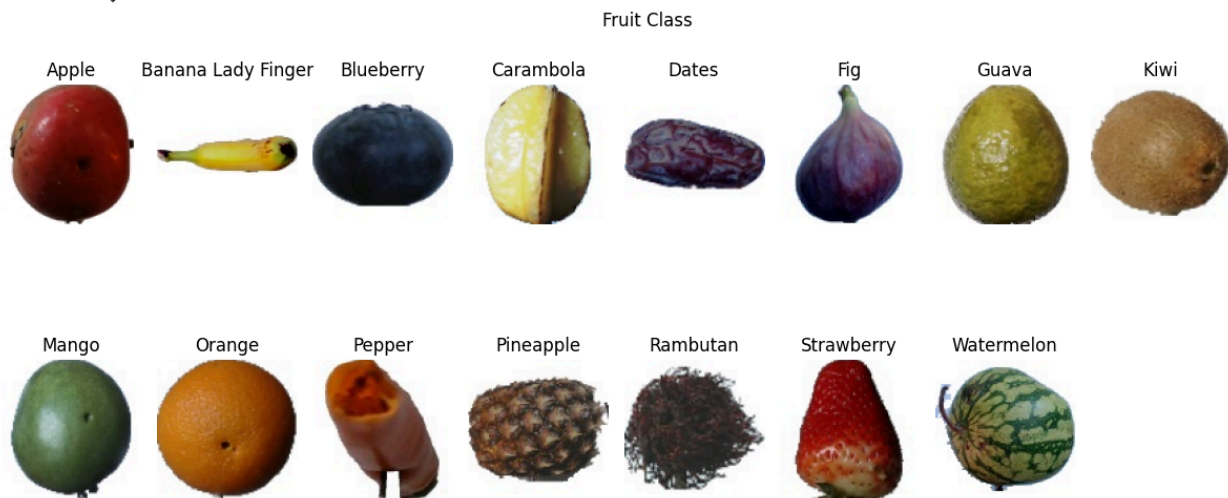
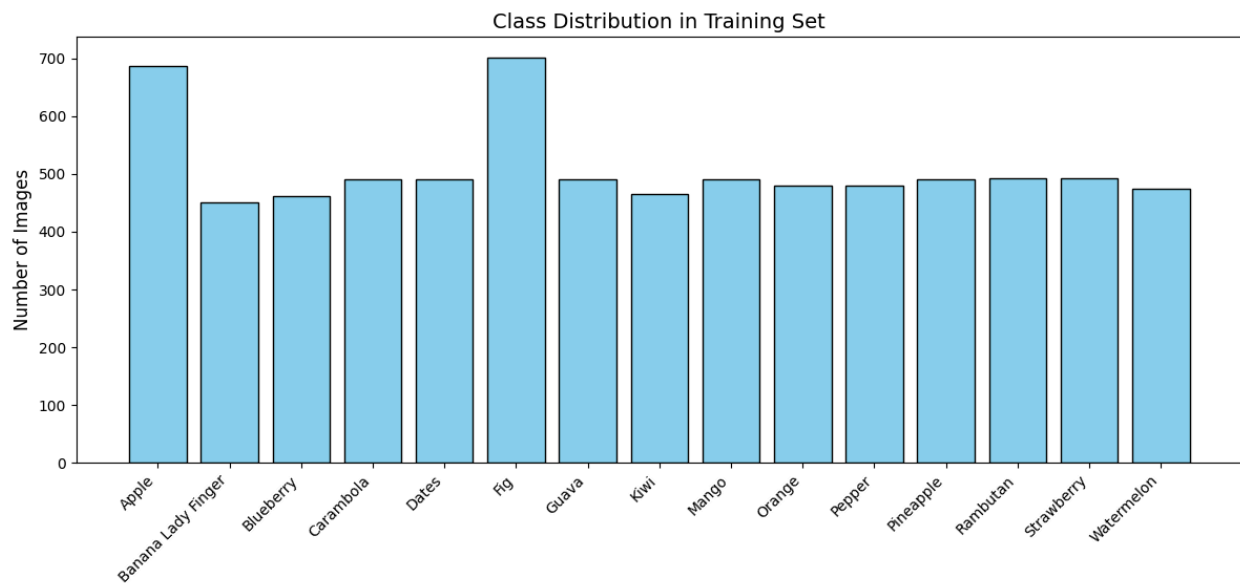
The dataset used in this project is a subset of the Fruits-360, publicly available in Kaggle which is authorized by Mihai Oltean. The dataset contains RGB images of different fruit classes captured in white background. Each image have 100 * 100 pixels.

The dataset contains 15 classes with the total number of images of 10,192 divided into training and test sets where training contains 7,635 images and Test contains 2,557 images.

Classes and their Image count:

SN no	Classes (Kaggle dir name)	Training Data	Test Data
1	Apple (Apple 8)	687	228
2	Banana (Banana Lady Finger 1)	450	152
3	Blueberry (Blueberry 1)	462	154
4	Carambola (Carambola 1)	490	166
5	Dates (Dates 1)	490	166
6	Fig (Fig 1)	702	234
7	Guava (Guava 1)	490	166
8	Kiwi (Kiwi 1)	466	156
9	Mango (Mango 1)	490	166

10	Orange (Orange 1)	479	160
11	Pepper (Pepper 2)	480	158
12	Pineapple (Pineapple 1)	490	166
13	Rambutan (Rambutan 1)	492	164
14	Strawberry (Strawberry 1)	492	164
15	Watermelon (Watermelon 1)	475	157

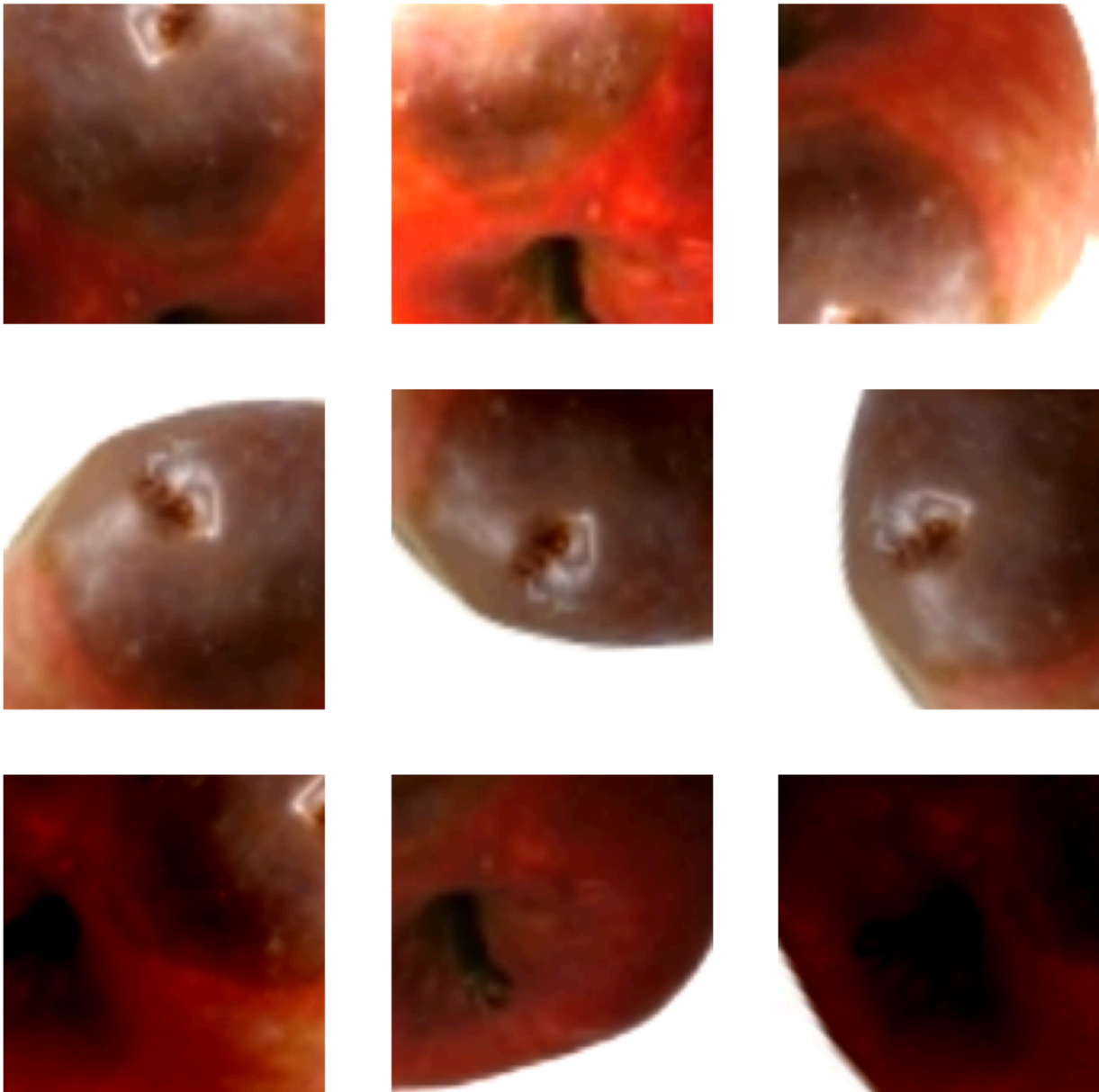


The dataset was divided into training set, validation set and test set which help us to evaluate models fairly.

In this project, Various data preprocessing techniques were used such as image resizing to 100*100 pixels, normalization using rescaling (1./255), label encoding using categorical representation and data augmentation using ImageDataGenerator.

Data augmentation techniques were used to increase the number of images and capping each class image to 1000. To create a diverse dataset and reduce overfitting, transformations such as rotation, horizontal flipping, zoom, width and height shifting and shearing were used. The augmentation technique helped to improve model generalization.

Sample Augmented Images



Methodology

Baseline CNN architecture

In this project, baseline CNN model was built on convolutional layers having 3 layers of convolutional with “relu” as an activation function and using MaxPooling layers after each convolution followed by 3 dense layers with “relu” activation function and last layer as output layer using softmax function.

In this architecture, Adam was used as optimizer, categorical crossentropy for loss function with 15 epochs, 32 batch size and default learning rate of Adam.

This model contained approximately 1.74 million trainable parameters.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 98, 98, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 49, 49, 32)	0
conv2d_4 (Conv2D)	(None, 47, 47, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_5 (Conv2D)	(None, 21, 21, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 10, 10, 128)	0
flatten_1 (Flatten)	(None, 12800)	0
dense_6 (Dense)	(None, 128)	1,638,528
dense_7 (Dense)	(None, 64)	8,256
dense_8 (Dense)	(None, 32)	2,080
dense_9 (Dense)	(None, 15)	495

Total params: 1,742,607 (6.65 MB)

Trainable params: 1,742,607 (6.65 MB)

Non-trainable params: 0 (0.00 B)

Deeper CNN Architecture

This is an extended version of baseline CNN architecture by adding more convolutional layers and increasing the number of filters. Deeper CNN architecture has additional convolutional layers, batch normalization, dropout layers and increased filter size.

By the use of batch normalization, the model is expected to stabilize training and improve convergence speed whereas using dropout layers model is to reduce overfitting by randomly disabling neurons during training.

The model used Adam and SGD optimizer to see which one performs better with the loss function of categorical crossentropy with 15 epochs and 32 batch size.

This model contained approximately 5.05 million trainable parameters highlighting model complexity.

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 100, 100, 32)	896
batch_normalization (BatchNormalization)	(None, 100, 100, 32)	128
conv2d_7 (Conv2D)	(None, 100, 100, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 100, 100, 32)	128
max_pooling2d_6 (MaxPooling2D)	(None, 50, 50, 32)	0
dropout_1 (Dropout)	(None, 50, 50, 32)	0
conv2d_8 (Conv2D)	(None, 50, 50, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 50, 50, 64)	256
conv2d_9 (Conv2D)	(None, 50, 50, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 50, 50, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 25, 25, 64)	0
dropout_2 (Dropout)	(None, 25, 25, 64)	0
conv2d_10 (Conv2D)	(None, 25, 25, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 25, 25, 128)	512
conv2d_11 (Conv2D)	(None, 25, 25, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 25, 25, 128)	512
max_pooling2d_8 (MaxPooling2D)	(None, 12, 12, 128)	0
dropout_3 (Dropout)	(None, 12, 12, 128)	0
flatten_2 (Flatten)	(None, 18432)	0
dense_10 (Dense)	(None, 256)	4,718,848
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dropout_4 (Dropout)	(None, 256)	0
dense_11 (Dense)	(None, 128)	32,896
dense_12 (Dense)	(None, 64)	8,256
dense_13 (Dense)	(None, 32)	2,080
dense_14 (Dense)	(None, 15)	495

Total params: 5,052,399 (19.27 MB)

Trainable params: 5,050,991 (19.27 MB)

Non-trainable params: 1,408 (5.50 KB)

Transfer Learning Model

Transfer learning was implemented by using a pre-trained model named MobileNet on ImageNet. To use this model, MobileNet was loaded with ImageNet weights. Then, removed the original classification head and added custom dense layers for 15-class classification. Initially, the convolutional base was frozen for feature extraction and later unfroze the selected layers for fine tuning. Smaller learning rate was used during fine-tuning to avoid catastrophic forgetting and images were resized to fit MobileNet requirements.

Model: "sequential_5"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 3, 3, 512)	14,714,688
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
dense_17 (Dense)	(None, 512)	262,656
batch_normalization_9 (BatchNormalization)	(None, 512)	2,048
dropout_5 (Dropout)	(None, 512)	0
dense_18 (Dense)	(None, 256)	131,328
batch_normalization_10 (BatchNormalization)	(None, 256)	1,024
dropout_6 (Dropout)	(None, 256)	0
dense_19 (Dense)	(None, 15)	3,855

Total params: 15,115,599 (57.66 MB)

Trainable params: 399,375 (1.52 MB)

Non-trainable params: 14,716,224 (56.14 MB)

Experiments and Results

Baseline vs Deeper Architecture

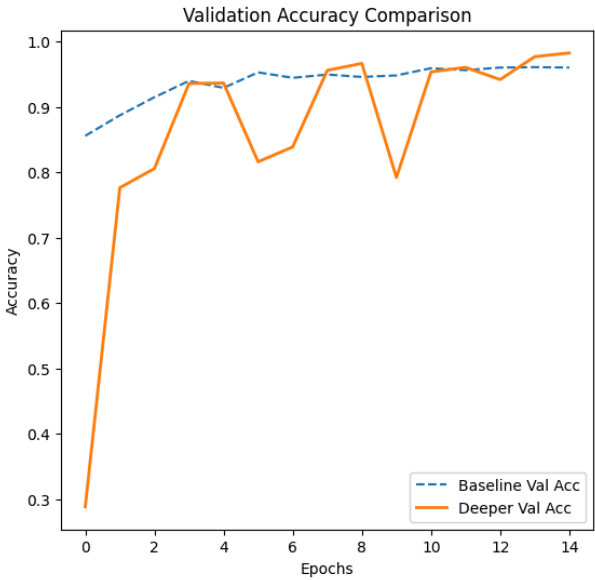
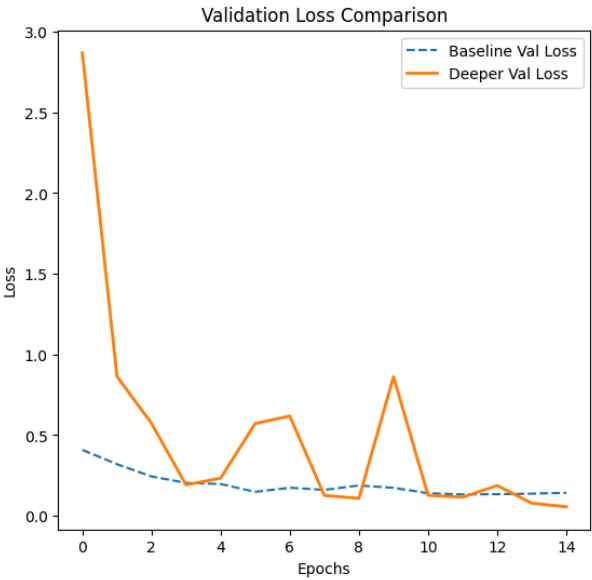
The baseline model achieved strong classification performance with validation accuracy of 94.9%, validation loss of 0.18 and training time of 82 seconds. The observation shows that the model learned efficiently even though it is simple architecture, minor cases of overfitting and visually similar fruit classes caused occasional misclassifications. This model achieved high precision, recall and F1-score across most fruit classes.

The deeper model had improved classification performance with results of 96% of validation accuracy, 0.11 of validation loss and training time of approximately 226 seconds. The observation shows this model required more computational resources to train, converged slowly in initial and overall generalization was improved

Comparison Between Baseline and Deeper Architecture

Metric	Baseline CNN	Deeper CNN
Parameters	1.74 Million	5.05 Million
Validation Accuracy	94.9%	96.0%
Training Time	82 sec	226 seconds
Overfitting	Moderate	Lower
Generalization	Good	Better

Adding more layers and features helped to improve feature extraction and classification accuracy. Despite being better, a deeper model required more computational and training time.



Computational Efficiency

The deeper CNN model had better accuracy with the increased computational complexity. For this project, I used a Kaggle notebook with T4 x2 GPU acceleration.

Aspect	Baseline Model	Deeper Model
Speed	Faster	Slower
Complexity	Lower	Higher
Accuracy	Lower	Higher
Resource Usage	Lower	Higher

The observation clearly shows, the baseline model is fast and computationally efficient whereas the deeper model has better performance.

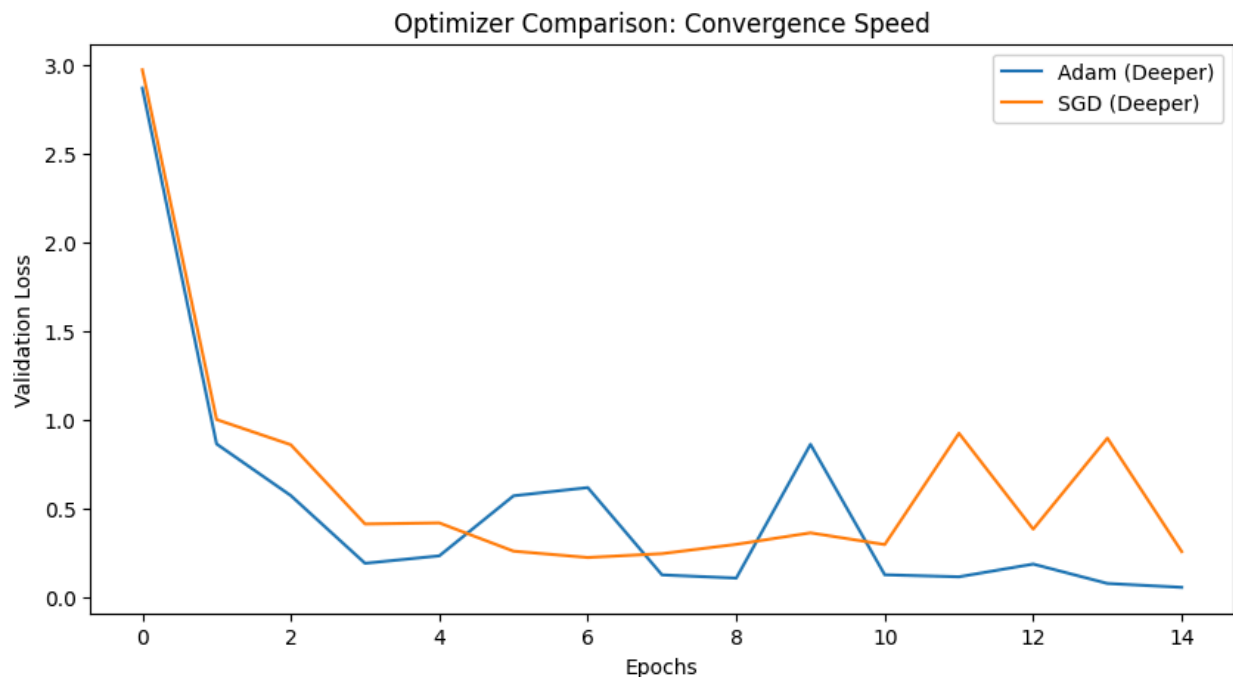
Optimizer Analysis: SGD and Adam

In this project, a deeper CNN model was trained using both SGD and Adam optimizers to compare performance. The observation showed, Adam optimizer is fast, better validation accuracy, lower validation loss, stable training behaviour with shorter training time.

As for the SGD optimizer, the observation showed slower convergence, lower final accuracy compared to Adam, higher validation loss, more fluctuations during training with longer training time.

Optimizer	Validation Accuracy	Validation Loss	Training time
Adam	96.0%	0.11	226 sec
SGD	94.8%	0.21	248 sec

At last, we can see Adam optimizer performed better overall due to adaptive learning rates, lower validation loss and fast convergence speed.



Ablation Study

It was done by removing dropout layers from deeper models. The result shows the validation accuracy dropped to 93.8%, validation loss increased to 0.27 with less training time without

dropout. The key observation shows overfitting increased noticeably, validation loss became unstable and dropout improved model generalization.

Model Config	Validation Accuracy	Validation Loss	Training Time
With dropout	96.0%	0.11	226 sec
Without dropout	93.8%	0.27	210 sec

By this experiment in the project, we know dropout plays a vital role to prevent overfitting in deeper CNN architecture. With removing dropout, it did reduce the training time, but the validation performance decreased. This indicates that regularization techniques are essential for maintaining strong performance.

Model	Val Accuracy	Val Loss	Training Time
Baseline CNN	96.00%	0.1428	78s
Deeper CNN (Adam)	98.23%	0.0564	222s
Deeper CNN (SGD)	94.57%	0.2579	211s
Ablation (No Dropout)	94.43%	0.2512	~60s
Transfer Learning (VGG16)	100%	0.6282	433s (Total)

Challenges in Training

Various challenges were encountered while training a data,

Overfitting: Deeper models were overfitted in the initial phase to increase complexity, To reduce this issue methods like dropout, batch normalization and data augmentation were used.

Computational Cost: Training deeper models required more computational resources.

Hyperparameter tuning

To implement appropriate learning rates and regularization parameters, multiple experiments was required.

Model Name	Time
Baseline Model	82 sec
Deeper Model	226 sec
Transfer Learning Model	Longer due to fine-tuning

Fine-Tuning and Transfer learning

To complete the project, I chose MobileNet due to its lightweight architecture, fast training speed, strong feature extraction capability and good performance on image processing and classification tasks. In this model, feature extraction was used. The convolutional base was frozen and only newly added classification layers were trained. With this in choice, observation showed it was faster in training, reduced computational requirements and prevented overfitting on small datasets.

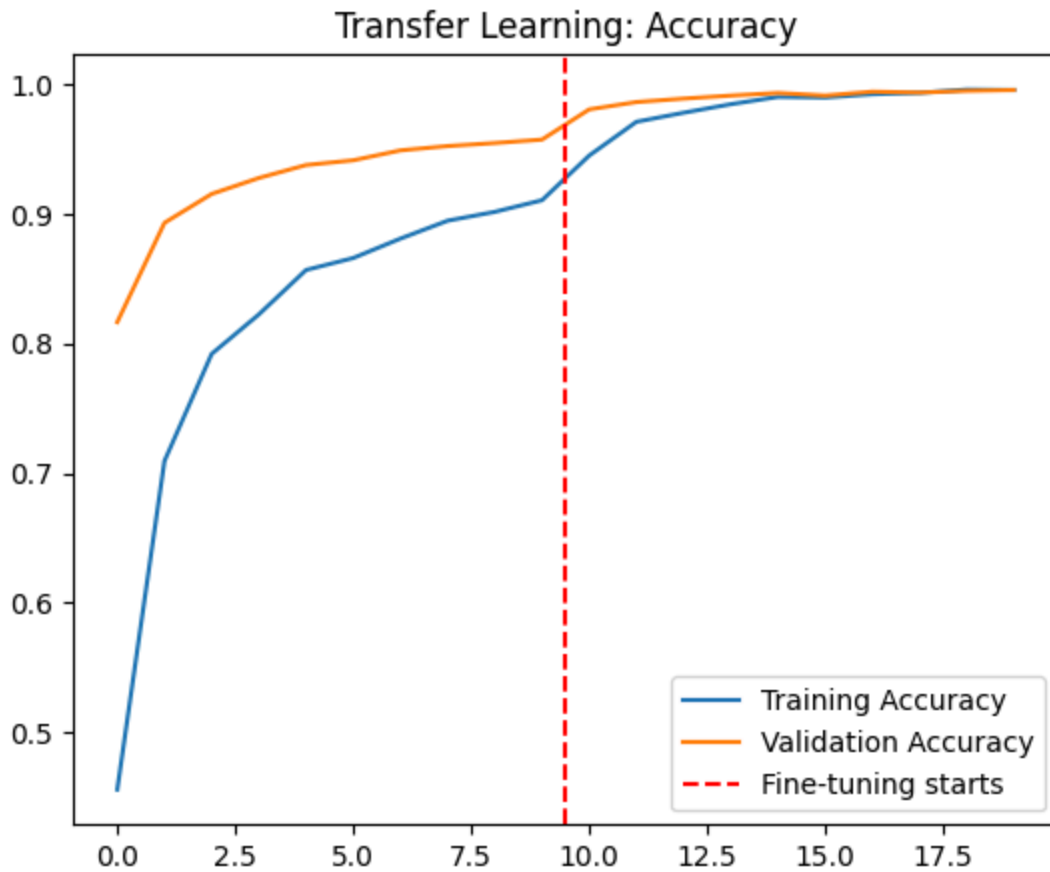
After feature extraction, selected convolutional layers were unfrozen for fine tuning. Implementation to lower learning rate, fine-tuned upper layers of MobileNet and improved adaption to fruit dataset.

The final performance of transfer learning with fine tuned parameters resulted in validation accuracy of 97.63% and final test accuracy of 100%.

Comparison

Model	Validation Accuracy
Baseline CNN	94.9%
Deeper CNN	96.0%
Transfer Learning	97.63%

From the observation, we can see transfer learning achieved the best performance. Pre-trained features significantly improved generalization and fine-tuning enhanced classification accuracy.



Conclusion

The project showed the building, training and evaluation of CBB architectures for fruit classification. The baseline CNN provided strong performance with low computational cost whereas the deeper CNN architecture improved classification accuracy through additional layers and regularization techniques.

Transfer learning using MobileNet achieved the best overall performance, demonstrating the effectiveness of pre-trained models for image classification tasks. The experiments also showed that Adam optimizer converged faster and produced better results than SGD.

Limitations

- Deeper models required higher computational resources.
- Some fruit classes remained visually challenging.
- Training required careful hyperparameter tuning.

Future Improvements

- Using larger datasets.
- Applying advanced architectures such as ResNet or EfficientNet.
- Performing hyperparameter optimization.
- Implementing ensemble learning techniques.
- Deploying the model as a web or mobile application.

Overall, the project highlights the effectiveness of CNNs and transfer learning for real-world image classification problems.